


ФГБОУ ВПО КубГУ

Курс лекций

2026 г

# Технологии программирования
# 5 сем. Язык JAVA

Зав. каф. ВМиИ, к.ф.-м.н,
доцент, Фоменко С.И.

## Тема 2: ООП в Java.
## Наследование и полиморфизм

# Концепция ООП

**Фундаментальные концепции:**

1. **Инкапсуляция;**
2. **Наследование;**
3. **Полиморфизм;**






# 1. Наследование: общие положения

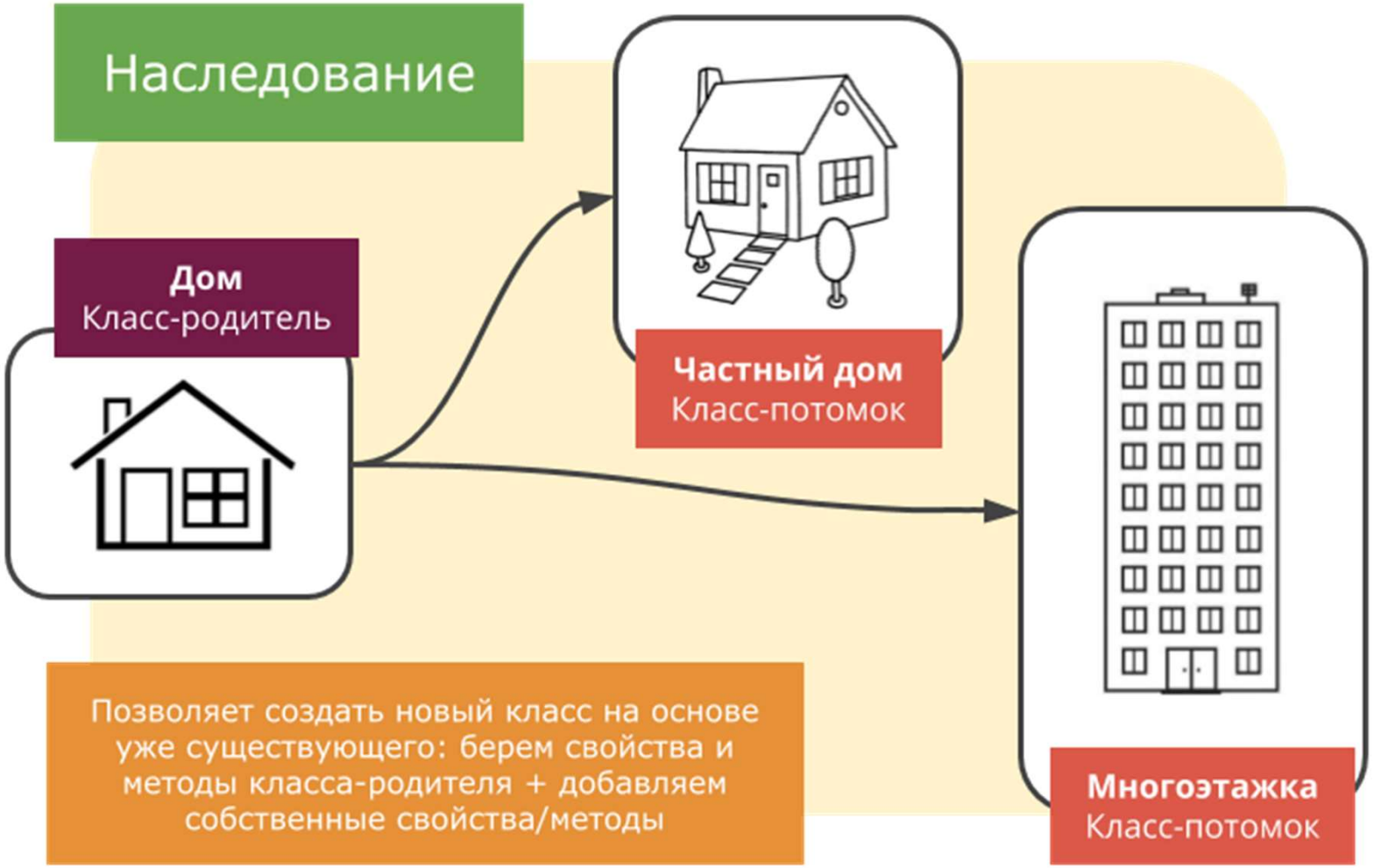
**Наследование** - способ создания нового класса на основе уже существующего, при котором класс-потомок заимствует свойства и методы родительского класса и также добавляет собственные.

Это дает возможность создавать абсолютно новые классы путем абстрагирования общих атрибутов и поведений.

**Суперкласс, или родительский класс** (также называемый **базовым**), содержит все атрибуты и поведения, общие для классов, которые наследуют его.

**Подкласс, или дочерний класс**, представляет собой расширение суперкласса.

# 1. Наследование: общие положения




https://smartiqa.ru/courses/python/lesson-6




# 1. Наследование: общие положения

**Для чего нужно наследование?**

- Дает возможность использовать код повторно. Классы-потомки берут общий функционал у родительского класса.
- Способствует быстрой разработке нового ПО на основе уже существующих открытых классов.
- Наследование позволяет делать процесс разработки проекта более гибким.

# 2. Наследование в Java

**Используется служебное слово extends (расширить)**

```
class A // суперкласс
  {
     …
  };

class B extends A  // подкласс B суперкласса A
  {
     …
  };
```

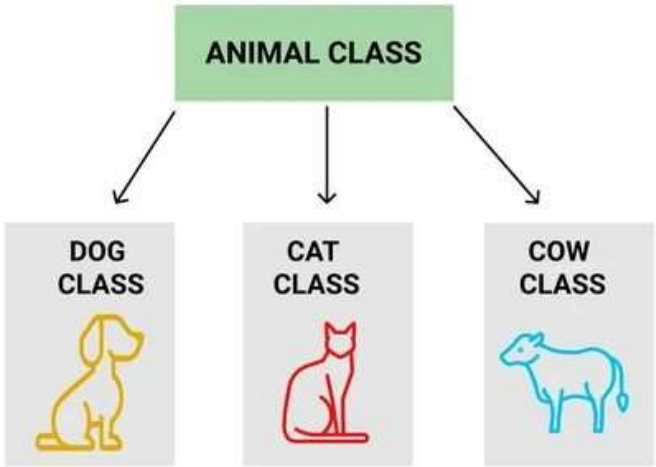
**класс A**

Поля класса A

**класс B**

Поля класса A

Поля класса B



## Запрет наследования

```
class A // суперкласс
  {
     …
  };

final class B extends A  // финализированный подкласс B
 суперкласса A
  {
     …
  };

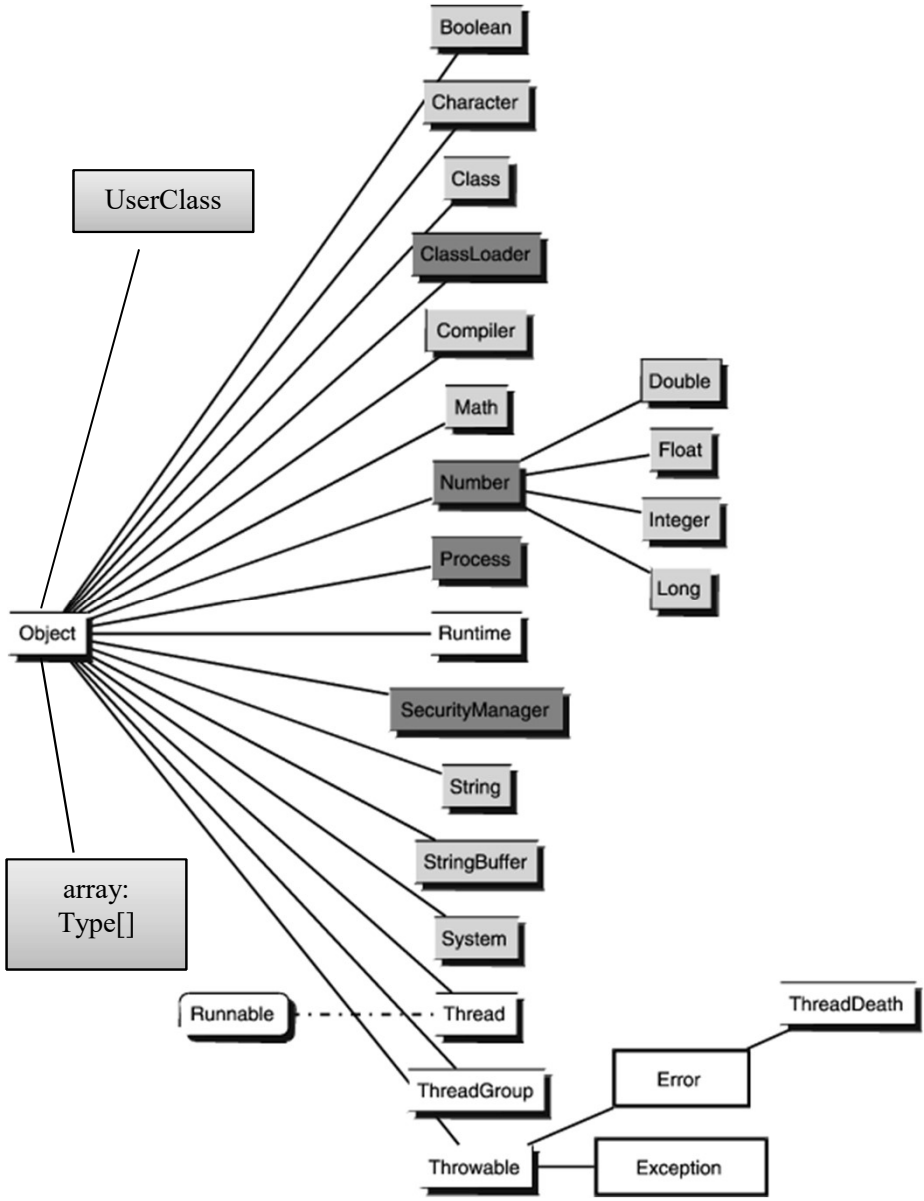
final class C // финализированный класс C
  {
     …
  };
```

# 3. Класс Object и иерархия стандартных классов

Класс **java.lang.Object** – это вершина иерархии классов в Java. Любой класс в языке (включая пользовательские и стандартные вроде String) является прямым или косвенным наследником Object.

Если при создании класса не указать ключевое слово extends, Java автоматически добавит наследование от Object.

```
// Вы пишете:
public class User { … }
// Java видит это как:
public class User extends Object { …}
```



# 3. Класс Object и иерархия стандартных классов

11 методов класса Object можно разделить на четыре группы.

1. Методы сравнения и жизненного цикла (Переопределяемые)

boolean equals(Object obj) – проверяет объекты на логическое равенство. По умолчанию (в классе Object) сравнивает ссылки (адреса в памяти) через ==. При переопределении разработчики пишут логику сравнения полей.

int hashCode() – возвращает целочисленный "хэш-код" объекта. Используется хэш-таблицами (HashMap, HashSet). Рекомендуется переопределить, если переопределен equals().

String toString() – возвращает строковое представление объекта. По умолчанию выводит строку вида ИмяКласса@хэшкод_в_шестнадцатеричном_виде.

2. Служебные методы (Обычно не переопределяются)

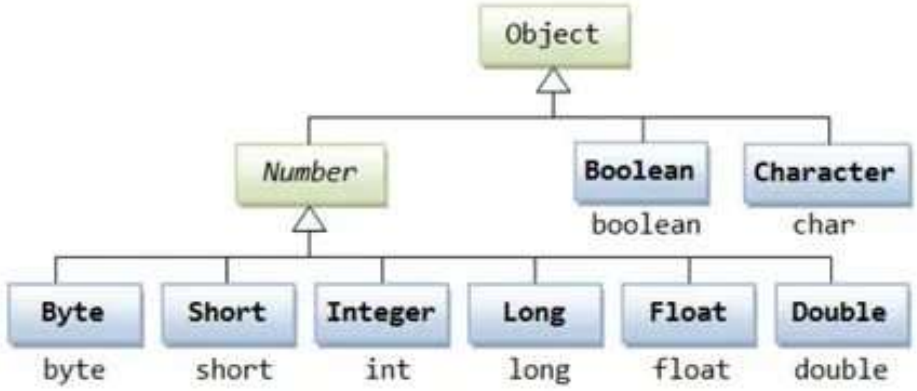
Class<?> getClass() – возвращает объект типа Class, содержащий метаданные о текущем классе (имя, методы, поля). Это фундамент механизма Reflection (рефлексии). Метод объявлен как final, его нельзя переопределить.

Object clone() – создает и возвращает копию объекта. Метод имеет модификатор protected. Работает только в том случае, если класс реализует интерфейс-маркер Cloneable, иначе выбросит ошибку CloneNotSupportedException. В современном коде часто заменяется конструкторами копирования.

3. Методы многопоточности (Синхронизация потоков). Будут рассмотрены позже.

# 4. Классы-обертки для примитивных типов

В Java для каждого примитивного типа (**не объект**) есть соответствующий ему объектный класс-обертка, где единственным атрибутом будет переменная соответствующего типа.




Java автоматически оборачивает примитивный тип там, где требуется тип Object

```
public void printObject(Object obj) { ... }
printObject(42); // Автоупаковка int в Integer
```



# 4. Классы-обертки для примитивных типов

Java автоматически распаковывает при математических и логических операциях с примитивными типами

```
int primitive = 10;
Integer object = 20;
int result = primitive + object;
if (primitive == object) {…}
```

**Но**, если сравниваются обертки, то

1) == -- сравниваются адреса , необходимо использовать метод equals

2) с другими логическими операциями происходит распаковка

```
Integer num1 = 200, num2 = 200, num3 = 201;
System.out.println(num1 == num2);  // FALSE!
System.out.println(num1.equals(num2)); // True!
System.out.println(num3>num1); // True!
```



# 5. Класс String

## Основные характеристики

1) Неизменяемость (immutable). Значения объектов String нельзя изменить после создания. Любая операция, которая модифицирует строку (например, изменение регистра), на самом деле создаёт новый объект String с нужным содержимым.
2) Финальный класс (final). Класс String объявлен как final, поэтому от него нельзя наследоваться.
3) Пул строк (String Pool). Java оптимизирует работу со строковыми литералами: если в пуле уже есть строка с таким же значением, программа просто переиспользует ссылку на существующий объект, а не создаёт новый.

## Создание строк

1) С помощью строкового литерала (распространённый способ):

```
String greeting = "Привет, мир!";.
```

2) С помощью конструктора. Например:

```
String str = new String("abc");
char chars = {'a', 'b', 'c'};
String s = new String(chars);.
```



# 5. Класс String

**Основные характеристики**

1) Неизменяемость (immutable). Значения объектов String нельзя изменить после создания. Любая операция, которая модифицирует строку (например, изменение регистра), на самом деле создаёт новый объект String с нужным содержимым.
2) Финальный класс (final). Класс String объявлен как final, поэтому от него нельзя наследоваться.
3) Пул строк (String Pool). Java оптимизирует работу со строковыми литералами: если в пуле уже есть строка с таким же значением, программа просто переиспользует ссылку на существующий объект, а не создаёт новый.

**Создание строк**

1) С помощью строкового литерала (распространённый способ):

```
String greeting = "Привет, мир!";.
```

2) С помощью конструктора. Например:

```
String str = new String("abc");
char chars = {'a', 'b', 'c'};
String s = new String(chars);.
```




# 5. Класс String

## Класс String предоставляет множество методов для работы со строками:

| Группа методов | Методы | Описание |
|---|---|---|
| **Инспекция** | int length(),<br>boolean isEmpty(),<br>boolean isBlank() | Возвращают длину строки,<br>проверяют, пуста ли она, и<br>проверяют, состоит ли строка только из пробельных символов (Java 11+) |
| **Поиск** | int indexOf(String s),<br>int lastIndexOf(String s),<br>contains(CharSequence s),<br>boolean startsWith(String s),<br>boolean endsWith(String s) | Находят позицию подстроки, проверяют наличие подстроки, определяют начало/конец строки<br>String ∈ CharSequence |
| **Извлечение подстроки** | String substring(int beginIndex)<br>substring(int beginIndex, int endIndex) | Возвращает подстроку, начиная с<br>beginIndex<br>и до<br>endIndex - 1 |
| **Преобразование регистра** | String toUpperCase(),<br>String toLowerCase() | Возвращают новую строку с преобразованным регистром |



# 5. Класс String

## Класс String предоставляет множество методов для работы со строками:

| Группа методов | Методы | Описание |
|---|---|---|
| **Замена** | String replace(CharSequence olds, CharSequence news),<br>replaceAll(..) | Заменяет все вхождения подстроки или вхождения по регулярному выражению |
| **Удаление пробелов** | String trim(),<br>String strip()  (Java 11+) | Удаляет начальные и конечные пробелы;<br>strip()  учитывает Unicode |
| **Сравнение** | boolean equals(Object obj),<br>boolean equalsIgnoreCase(String s),<br>int compareTo(String s) | Сравнивают строки с учётом регистра,<br>без учёта регистра<br>выполняют лексикографическое сравнение:  0 (=), -1 (<), 1 (>) |
| **Разделение** | split(String regex) | Разделяет строку на массив подстрок по заданному регулярному выражению |
| **Форматирование** | static String format(String f, Object … args),<br>static String valueOf(T arg) | Создаёт форматированную строку из объектов;<br>преобразует тип T в строку |

# Концепции ООП

**Фундаментальные концепции:**

1. Инкапсуляция;
2. Наследование;
3. **Полиморфизм;**




**Дополнительные положения:**

4. Абстрагирование
5. Композиция (агрегация).




# 6. Полиморфизм

*Полиморфизм* — с греческого буквально «множественность форм».

**Полиморфизм** – это способность использование одной обобщенной процедуры для разных типов данных.

**Параметрический полиморфизм** – обработка значений разных типов идентичным образом.

**Ad-hoc полиморфизм** (с англ. специальный (для данного случая) полиморфизм) – это поддержка методов или процедур с общим именем. Достигается за счет перегрузки, т.е. новой реализации метода для нового набора входных данных.

**Полиморфизм подтипов** – объекты разных классов используются через общий интерфейс или базовый класс (наследование, интерфейсы)




# 6. Полиморфизм

В классическом ООП полиморфизм реализуется через

**1) Ad-hoc полиморфизм:**

- **перегрузку** методов – это статический полиморфизм, разрешаемый во время компиляции программы
- **переопределение** (override) наследуемых методов – это динамический полиморфизм, разрешаемый во время исполнения программы

**2) Полиморфизм подтипов**, когда базовый класс представляет своего наследника, и через него вызываются общие методы.



# 7. Полиморфизм при наследовании в Java

Все наследуемые методы могут быть переопределены.

**Требования**

1) Совпадение сигнатуры: в подклассе нужно полностью повторить последовательность имён параметров и их типов из метода суперкласса

2) Тип возвращаемого значения – ковариантный. В подклассе можно вернуть не только тот же тип, что и в родительском методе, но и его подтип.

3) Метод должен быть доступен для наследования. Нельзя переопределить:

- private методы – они  не видны в подклассе;
- static методы – они не участвуют в полиморфизме, а в наследнике происходит их сокрытие (hiding): фактически происходит замена;
- final методы – модификатор final явно запрещает переопределение

4) Можно менять уровень доступа, однако он должен только повышаться. Если в суперклассе метод public, в подклассе нельзя сделать его protected или private. Но protected в родителе допустимо расширить до public в наследнике



# 7. Полиморфизм при наследовании в Java

Рекомендации

1) Используйте аннотацию **@Override**. Она не обязательна, но настоятельно рекомендуется, т.к. при компиляции проверяется, что метод действительно переопределяет какой-то родительский.

2) Использование **super** для реализации метода родителя. Это распространённый паттерн: в переопределённом методе сначала вызывается super.method(...), а затем добавляется своя логика.

# 7. Полиморфизм при наследовании в Java

```
class C{
    protected int  a,b;
    public C(int a, int b) { this.a = a; this.b = b; }
    protected int calc() { return a+b; }
};
class D extends C {
  protected int  c;
  public  D(int a, int b , int c)
  {
    super(a,b); // вызов конструктора суперкласса
    this.c=c;
  }
 @Override
  public int calc() { return super.calc()*c; }
};
```

# Полиморфизм при наследовании в Java

```
class C{
    protected int  a,b;
    public C(int a, int b) { this.a = a; this.b = b; }
    protected int calc() { return a+b; }
};
class D extends C {
  protected int  c;
  public  D(int a, int b , int c)
  {
    super(a,b);
   this.c=c;
  }
 //@Override
  public int calc() { return super.calc()*c; }
};
```

# 7. Полиморфизм при наследовании в Java

Однако

```
class C{
    protected int  a,b;
    public C(int a, int b) { this.a = a; this.b = b; }
    public double calc2(double f) { return (a+b)*f; }
};
class D extends C {
  protected int  c;
  public  D(int a, int b , int c)   {   super(a,b);   this.c=c;   }
 //@Override
  public double calc2(int f) { return super.calc2(f)*c; }};


…
C[] arC={new C(1,2), new D(1,2,3)};
System.out.println(arC[0].calc2(1)); // 3
System.out.println(arC[1].calc2(1));     // ?
```

# 7. Полиморфизм при наследовании в Java

```
class C{
    protected int  a,b;
    public C(int a, int b) { this.a = a; this.b = b; }
    public double calc2(double f) { return (a+b)*f; }
};
class D extends C {
  protected int  c;
  public  D(int a, int b , int c)   {   super(a,b);   this.c=c;   }
 //@Override
  public double calc2(int f) { return super.calc2(f)*c; }}; // ошибка это не переопределённый, а перегруженный метод
…
C[] arC={new C(1,2), new D(1,2,3)};
System.out.println(arC[0].calc2(1)); // 3
System.out.println(arC[1].calc2(1)); // ожидаем 9, но 3
```

Однако

```
3.0
3.0
```

# 7. Полиморфизм при наследовании в Java

```
class C{
    protected int  a,b;
    public C(int a, int b) { this.a = a; this.b = b; }
    public double calc2(double f) { return (a+b)*f; }
};
class D extends C {
  protected int  c;
  public  D(int a, int b , int c)   {   super(a,b);   this.c=c;   }
 @Override
  public double calc2(int f) { return super.calc2(f)*c; }}; // ошибка это не переопределённый, а перегруженный метод
…
C[] arC={new C(1,2), new D(1,2,3)};
System.out.println(arC[0].calc2(1)); // 3
System.out.println(arC[1].calc2(1)); // ожидаем 9, но 3
```

Однако

```
3.0
3.0
```

# 8. Класс Point. Переопределение методов

```
class Point
{
 private  double x,y;
 public Point(double x0, double y0){ x=x0; y=y0;}
 public Point(double x0){this(x0,x0);}

 public double dist(Point b) {
   return Math.sqrt((x-b.x)*(x-b.x)+(y-b.y)*(y-b.y));
 }
  // переопределение методов из Object
 @Override
 public String toString() { return "("+x+","+y+")"; }
 @Override
  public boolean equals(Object obj) {
        if(obj instanceof Point) // проверка типа
          return ((Point)obj).x==x && ((Point)obj).y==y;
        else return false;
    }
};
...
Point P1=new Point(1, 2), P2=new Point(1, 1);
System.out.println("P1"+P1); System.out.println("P2"+P2);
System.out.println("P1==P2: "+P1.equals(P2));
```

```
P1(1.0,2.0)
P2(1.0,1.0)
P1==P2: false
```

# 8. Класс Point. Переопределение методов

```
class Point
{
 private  double x,y;
 public Point(double x0, double y0){ x=x0; y=y0;}
 public Point(double x0){this(x0,x0);}

 public double dist(Point b) {
   return Math.sqrt((x-b.x)*(x-b.x)+(y-b.y)*(y-b.y));
 }
  // переопределение методов из Object
 @Override
 public String toString() { return "("+x+","+y+")"; }
 @Override
  public boolean equals(Object obj) {
        if(obj instanceof Point) // проверка типа
          return ((Point)obj).x==x && ((Point)obj).y==y;
        else return false;
    }
};
...
Point P1=new Point(1, 2), P3=new Point(P1);
System.out.println("P1"+P1); System.out.println("P3"+P3);
System.out.println("P1==P3: "+P1.equals(P3)+" adrP1==adrP3:
                   (P1==P3) );
```

```
P1(1.0,2.0)
P3(1.0,2.0)
P1==P3: true
adrP1==adrP3: false
```

# 9. Абстрактный класс

Абстрактный класс объявляется с ключевым словом abstract.

Должен содержать хотя бы один абстрактный метод, т.е. чистую сигнатуру

```
abstract class A // абстрактный суперкласс
  {
     …
  abstract [префикс] тип имя_метода(параметры);
  };

class B extends A  // расширение суперкласса A
  {
      …
  };
```

# 9. Абстрактный класс

```
public abstract class Shape {
    protected String name;
    public Shape(String name) {
        this.name = name;
    }

    public abstract double area();

    @Override
    public String toString() {return "Имя: " + name + ", площадь: " + area();}

}

public class Rectangle extends Shape {
   protected double a, b;

   public Rectangle(double a, double b){ super("Rectangle"); this.a=a; this.b=b; }

   @Override
   public double area() {
        return a*b;
   }
}
```



# 9. Абстрактный класс

Ключевые цели абстрактных классов:

1) Задать обязательный контракт и при этом дать общую реализацию. У разных подклассов есть общая логика, но некоторые части должны быть реализованы по-разному.

2) Реализовать паттерн «Шаблонный метод»: в абстрактном классе фиксируют **порядок шагов алгоритма**, а конкретные шаги делегируют абстрактным методам.

```
public String toString() {return "Имя: " + name + ", площадь: " + area();}
```

3) Обеспечить полиморфизм и единый интерфейс для разных реализаций

```
Shape s = new Rectangle(1,2);
System.out.println(s); // работает, хотя реализация area()в Rectangle
```

4) Устранить дублирование кода и повысить поддерживаемость



# 10. Интерфейс

Интерфейс -- набор объявлений методов (а также констант), которые класс, который воплощает данный интерфейс, обязуется реализовать. Т.е. – это контракт. Интерфейс описывает, что объект может делать, но не как.

Ключевые слова: **interface – объявление, implements – реализация** в классе

```
[префикс] interface I
 {
   тип  имя_константы = value; // необязательные константы
   [префикс] тип имя_метода(параметры); // сигнатура метода
 };

class B [extends C] implements I  // реализация
 интерфейса I
 {
    …
 };
```



# 10. Интерфейс

Если обычное наследование должно быть единственным, то наследование интерфейса может быть множественным

```
class B extends C implements I1,I2  // наследование C,
 реализация интерфейсов I1,I2
 {
    …
 };
```

Класс, который принимает интерфейс должен обязательно реализовать методы интерфейса.

# 10. Интерфейс

```
public interface IMesure {
 double mes();
}

public abstract class Shape {
    ...
     public abstract double area();
     @Override
     public double mes() {    return area();  }
     ...
}

abstract class Body implements IMesure  {
    ...
     public abstract double volume();
     @Override
     public double mes() { return volume(); }
    ...
}
 ...

 IMesure[] M = {new Rectangle(1,2); new Cube(1,2,3)}; //интерфейсная переменная-массив
 for(var m:M) {System.out.println(m.mes());}
```



# Абстрактный класс или интерфейс?

Нужна общая реализация и/или шаблонный алгоритм, и/или есть иерархия («является разновидностью чего-то») => **абстрактный класс**

Нужен чистый контракт (сервис) без привязки к реализации => **интерфейс**




# Литература

1. Г. Шилдт, Java. Полное руководство, 12-е изд. : Пер. с англ. - СПб. "Диалектика"; 2023. - 1344 с.